RECONAV: RELIABLE CONSTRAINED LONG-HORIZON VISION-AND-LANGUAGE NAVIGATION VIA A DETERMINATION ENGINE

Anonymous authors
Paper under double-blind review

ABSTRACT

Practical navigation demands more than visiting a sequence of targets over a long horizon: agents must also respect hard and soft constraints over which targets to visit and which routes to take. We study this setting as constrained long-horizon vision-and-language navigation (VLN), where constraints may apply to individual stages or the entire journey. Existing memory- or map-based navigation agents support long-horizon planning, while planners can plan routes under constraints correctly extracted from instructions and grounded in the scene. Yet existing methods do not reliably propagate scoped constraints across instruction interpretation, scene grounding, and complete-route planning, causing failures in target resolution and constraint satisfaction.

To evaluate this gap, we first construct a benchmark of 192 scene-grounded tasks across nine indoor scenes, designed to measure joint target and route satisfaction. Existing methods achieve low satisfaction even with global scene information. We then introduce ReCoNav, whose instruction contract jointly represents visits, their order, and scoped hard and soft constraints for target and route selection. Its Determination Engine propagates constraints across interpretation, grounding, and planning, using scene and task rules to derive and verify judgments, while vision-language models (VLMs) make the remaining semantic judgments and repair erroneous ones using evidence from the engine. On the full benchmark, ReCoNav achieves 81.25% full-instruction satisfaction while maintaining low mean token usage per completed run. The strongest evaluated existing method with global scene information achieves 24.48%, and direct planning with feedback using the same scene information, VLM backbone, and solver achieves 44.27%. Ablations across four VLM backbones support the design of the Determination Engine.

1 INTRODUCTION

Conventional vision-and-language navigation (VLN) asks a navigation agent to follow a language description of a route to a destination (Anderson et al., 2018; Jain et al., 2019; Krantz et al., 2020; Ku et al., 2020). Recent work studies long-horizon VLN through sequences of targets or navigation subtasks (Huang et al., 2023; Song et al., 2025; Zhu et al., 2025; Han et al., 2026). Practical navigation instructions can additionally impose hard and soft constraints on target and route selection. We study *constrained long-horizon VLN*, where these constraints may apply to individual stages or the entire journey.

Figure 1 illustrates these three settings in the same scene. The route-following instruction describes movements through `Dining` to a `bed`. The long-horizon instruction specifies visiting `Dining` and then a `bed`, leaving the choice of bed and the routes open. The constrained instruction adds the hard constraint `at most two rooms` and the soft constraint `try to stay away from the rug` for the entire route from `Dining` to the selected bed. The route through `Kitchen` passes near the rug, violating the soft constraint. Reaching `Bed A` or `Bed B` through the bedroom containing `Bed C` and the hallway exceeds the two-room limit. The illustrated route to `Bed C` satisfies both constraints.

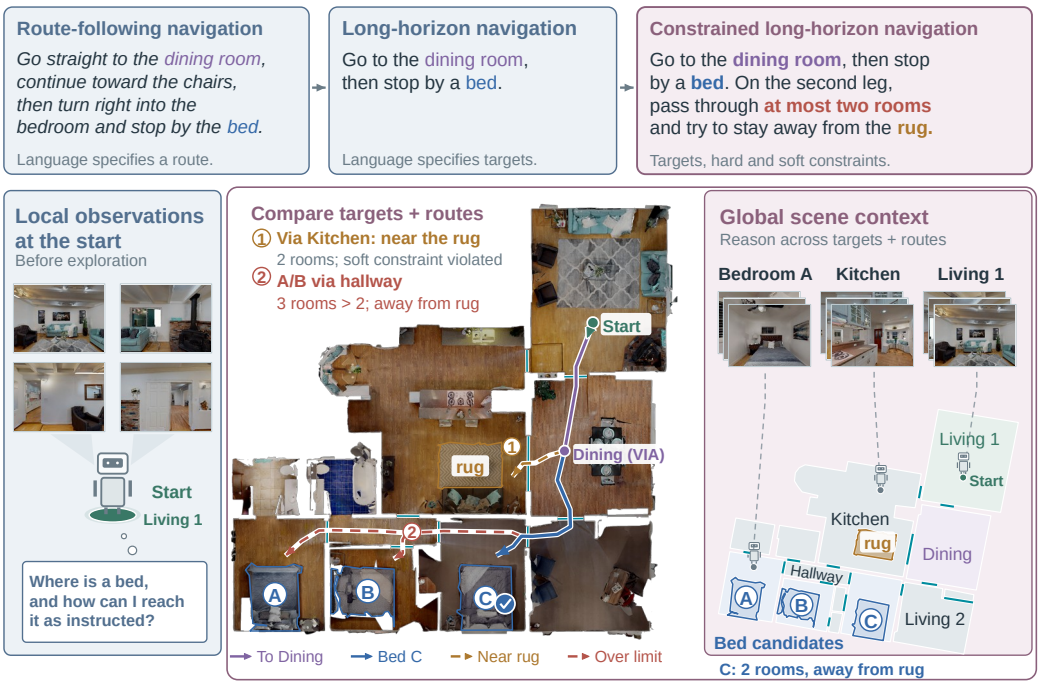


Figure 1: **From route following to constrained long-horizon navigation.** All three instructions guide the agent from `Living 1` to `Dining` and then to a `bed`. For the constrained instruction, `at most two rooms` and `try to stay away from the rug` apply throughout the route from `Dining` to the selected bed. Local observations show only the agent's immediate surroundings, while global scene context reveals candidate beds (`A`–`C`), room connectivity, and route alternatives for comparison under these constraints. The illustrated route to `Bed C` satisfies both constraints.

Local observations alone may not reveal all the scene information needed to satisfy the constraints. Memory-based navigation agents extend the available context through visual histories and task memories (Zhang et al., 2024; Song et al., 2025). Map-based agents provide broader scene information, including through pre-exploration (Huang et al., 2023; Bhatt et al., 2026; Zhang et al., 2026). Planners can plan routes under constraints correctly extracted from instructions and grounded in the scene (Liu et al., 2024; Quartey et al., 2025). However, reliable full-instruction satisfaction requires accurately extracting constraints and their scopes from instructions and applying them consistently throughout scene grounding and complete-route planning. For example, in Figure 1, global scene context reveals multiple candidate beds, but selecting among them requires evaluating routes to each candidate under the instruction's constraints.

To evaluate agents on constrained long-horizon VLN, we construct a **benchmark of 192 scene-grounded tasks across nine indoor scenes**. Tasks combine visits, their order, and scoped hard and soft constraints. On this benchmark, the best full-instruction satisfaction among evaluated existing methods is **8.43%** with their original interfaces and **24.48%** with global scene information (Table 2). These results show that even with global scene information, evaluated methods struggle to satisfy the full instruction.

We then introduce ReCoNav, whose *instruction contract* jointly represents visits, their order, and scoped hard and soft constraints for target and route selection. An *Affordance Scene Graph* (ASG) provides global scene information, including entities, spatial relations, room connectivity, and traversability, for grounding constraints and evaluating complete routes. Our insight is that semantic judgments made by vision-language models (VLMs) may be incorrect, but scene and task rules can determine their consequences and reveal conflicts among them. Based on this insight, we design a *Determination Engine* that propagates constraints across instruction interpretation, scene grounding, and complete-route planning. VLMs make the remaining semantic judgments and use evidence

from the engine to repair erroneous ones. After a repair, the engine updates and rechecks all affected judgments.

ReCoNav achieves **81.25% full-instruction satisfaction**, compared with 44.27% for direct planning with feedback using the same ASG, VLM backbone, and solver. It uses an average of 7.12k model tokens per completed run, compared with 113.15k for this feedback baseline (Table 2). Ablations across four VLM backbones support the design of the Determination Engine.

Our contributions are (i) a scene-grounded benchmark for constrained long-horizon VLN; (ii) an instruction contract jointly representing visits, their order, and scoped hard and soft constraints for target and route selection; and (iii) ReCoNav, whose Determination Engine uses scene and task rules to derive and verify judgments and guide VLM repair.

2 Related Work

Navigation and scene representations. Navigation history and memory help agents retain information from earlier observations and visits (Zhou et al., 2024; Song et al., 2025; Zhu et al., 2025; Han et al., 2026). Maps organize scene information for planning. Topological maps represent connectivity (Chen et al., 2022; 2024), while semantic maps and scene graphs provide object and spatial information (Zhang et al., 2025; Huang et al., 2023; Gu et al., 2024; Werby et al., 2024; Li et al., 2026). Pre-exploration builds scene representations before execution to provide global scene information (Zhang et al., 2026; Bhatt et al., 2026). ReCoNav uses an ASG to represent global scene information for grounding constraints and evaluating complete routes.

Constraints in navigation. Navigation instructions can specify spatial relations and movement conditions. NavSpace evaluates spatial instruction following, and its SNav baseline is trained on instructions that require spatial perception and reasoning (Yang et al., 2026). CA-Nav uses constraints to determine when a subinstruction is complete and to guide navigation (Chen et al., 2025). GC-VLN translates route descriptions into geometric constraints among waypoints and objects to determine waypoint positions, retaining alternative solutions for backtracking (Yin et al., 2025). In ReCoNav, scoped hard and soft constraints, including avoidance and counts of distinct entities, jointly govern target selection and complete-route planning.

Language models for reasoning and planning. SatLM and LLMFP combine language models with formal solvers for reasoning and planning (Ye et al., 2023; Hao et al., 2025). SayPlan uses feedback from a scene graph simulator to revise robot plans (Rana et al., 2023). Lang2LTL-2 and LIMP use grounded linear temporal logic (LTL) specifications to constrain robot plans (Liu et al., 2024; Quartey et al., 2025). Inspired by these approaches, we design an instruction contract that jointly represents visits, their order, and scoped hard and soft constraints. ReCoNav uses this contract with scene and task rules to derive and verify judgments and guide VLM repair across instruction interpretation, scene grounding, and complete-route planning.

3 Method

3.1 Instruction Contract

Given registered RGB-D observations of a static scene, an instruction u, and a start pose s_0, the task is to produce a traversable, collision-free waypoint route $\pi = (s_0, q_1, \ldots, q_n)$ satisfying u. The instruction contract $\mathcal{U} = (E, O, H, P)$ jointly represents visit events (E), their order (O), scoped hard constraints (H), and soft constraints (P) that express preferences. Let N be the set of scene entities and $D(v)$ the visit area of $v \in N$: an object's neighborhood, a region or level interior, or a portal interface.

Visits and order. A realization ω consistently assigns each realized visit event $e_i \in E$ a matching target $v_i \in N$ and a waypoint index $j_i \in \{0, \ldots, n\}$, written as $\omega(e_i) = (v_i, j_i)$. With $q_0 = s_0$, the visit condition and the condition for the required final visit event $e_g \in E$ are

$$\mathrm{VIA}(e_i):\ q_{j_i} \in D(v_i), \qquad \mathrm{GOAL}(e_g):\ j_g = n,\ q_n \in D(v_g). \tag{1}$$

Portal visits additionally require crossing the interface. Every other realized event precedes e_g, so $j_i < j_g$ for $e_i \neq e_g$. Only preferred visits may remain unrealized. Repeated visits to the same entity remain distinct events. For realized events, a precedence constraint $e_i \prec e_j$ in O requires $j_i < j_j$. BEFORE and AFTER use this strict order. NEXT and BETWEEN additionally restrict intervening requested events (Appendix A).

Scoped constraints. The hard constraints $H = \mathcal{A} \cup \mathcal{C}$ comprise scoped avoidance constraints $\mathcal{A}$ and count constraints $\mathcal{C}$. For a scope $\sigma = (e_a, e_b)$ with $j_a < j_b$, let $N_\sigma(\pi, \omega)$ be the set of distinct entities encountered along π after event e_a and before event e_b. This includes entities encountered while moving between waypoints. For entity sets $N_A, N_C \subseteq N$ and integer $k \geq 0$,

$$\text{AVOID}(N_A, \sigma): \quad N_\sigma(\pi, \omega) \cap N_A = \varnothing, \tag{2}$$

$$\text{COUNT}_{\bowtie k}(N_C, \sigma): \quad |N_\sigma(\pi, \omega) \cap N_C| \bowtie k, \quad \bowtie \in \{\leq, =, \geq\}. \tag{3}$$

An encounter is an entry into $D(v)$, except that portals require crossing the interface. Each entity is counted at most once per scope, even if encountered repeatedly. The route start and final visit may also serve as scope boundaries. Appendix A details boundary conventions.

Preferences and objective. Soft constraints in P use the same visit, order, avoidance, and count predicates, with positive costs for violations. Let $p(\pi, \omega; P)$ denote the total violation cost. Let $\mathcal{F}(\mathcal{U})$ contain all pairs (π, ω) for which π is traversable and collision-free and the required visits, applicable orders, and hard constraints are satisfied. Joint target and route selection minimizes this cost over the feasible pairs:

$$(\pi^*, \omega^*) \in \arg \min_{(\pi,\omega) \in \mathcal{F}(\mathcal{U})} p(\pi, \omega; P). \tag{4}$$

3.2 ReCoNav Overview

Figure 2 presents the ReCoNav framework. ReCoNav constructs an ASG from pre-exploration RGB-D observations and extracts an instruction contract from u. The Determination Engine uses scene and task rules to propagate constraints across instruction interpretation, scene grounding, and complete-route planning, deriving and verifying judgments. VLMs make the remaining semantic judgments and use evidence from the engine to repair erroneous ones.

Affordance Scene Graph. The ASG $\mathcal{G} = (N, M, \Gamma)$ represents the global scene. Its entities N include labeled objects, regions, portals, and levels. Relations M encode object–region and region–level membership, together with connectivity through portals. Geometry Γ supports spatial tests and defines visit areas $D(v)$ for checking visits and encounters along routes. It also defines a navigation graph G_{nav} of navigable positions and traversable connections. We construct the ASG from registered RGB-D observations collected during pre-exploration, using 3D object instances from MaskClustering (Yan et al., 2024), VLM recognition, and region/portal reconstruction (Appendix C).

Instruction interpretation, grounding, and planning. The VLM extracts the instruction contract $\mathcal{U}$ from u, retaining the corresponding source phrases. Target descriptions retain spatial qualifiers such as *in*, *near*, and *nearest*, evaluated using ASG relations and geometry (Appendix A.1). For each visit event e_i, semantic matching and these spatial conditions define a candidate target set $N_i \subseteq N$. For example, $\text{IN}(e_i, n_y)$ requires the target of e_i to belong to region or level $n_y \in N$. A candidate entity $n_x \in N$ is excluded from N_i if $n_x \notin n_y$ (Figure 2). Semantic matching and spatial qualifiers also identify the entity sets N_A for AVOID and N_C for COUNT. The corresponding rules check avoidance and distinct-entity counts using the entities encountered within each scope. A candidate target binding $\beta : E \to N$ assigns each event a target $\beta(e_i) \in N_i$, with coreferent events assigned to the same target. In Figure 2, $\beta(v) = \text{R4}$ binds the bedroom visit v to region R4. For each realized event, $\omega(e_i) = (\beta(e_i), j_i)$. The engine queries the route solver both to evaluate candidate bindings and to construct complete routes. Preferences guide joint target and route selection. A route π is accepted only after replay verifies the required visits, applicable orders, and hard constraints under the grounded contract.

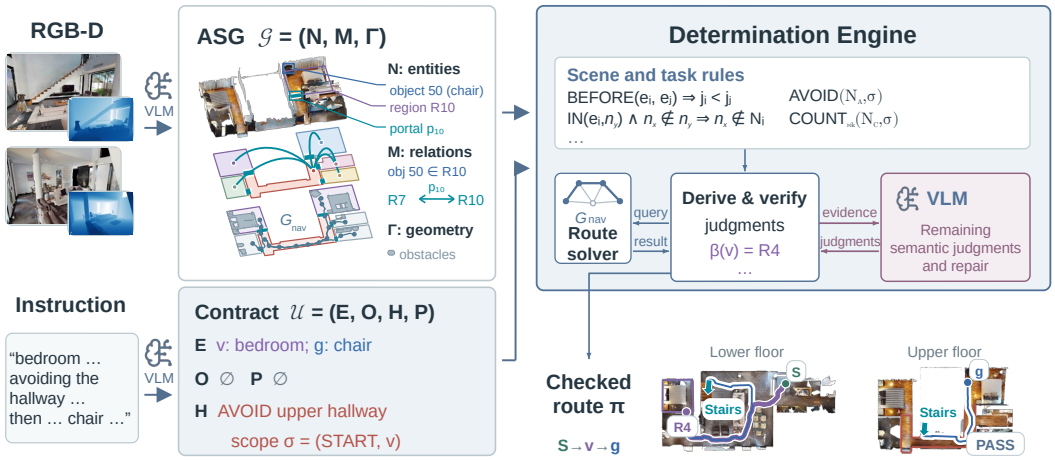


Figure 2: **ReCoNav overview.** RGB-D observations and the instruction are represented as an ASG and an instruction contract. The Determination Engine uses scene and task rules and route queries to derive and verify judgments. VLMs make remaining semantic judgments and use the engine's evidence for repair. The engine returns a checked route π. The ASG layers show entities N, relations M, and geometry Γ, which defines the navigation graph G_{nav}. The example rules enforce event order (BEFORE), filter candidate targets using spatial qualifiers (IN), and check scoped avoidance and counts (AVOID and COUNT). S, v, g denote the start, bedroom visit, and final chair visit. $\sigma = (\text{START}, v)$ specifies the avoidance scope. The judgment $\beta(v) = \text{R4}$ binds the bedroom visit to region R4.

3.3 Determination Engine

The engine maintains a set of judgments $\mathcal{K}$ and unresolved alternatives across instruction interpretation, grounding, and planning. Each judgment $K \in \mathcal{K}$ records a decision, such as a semantic label, event binding, order, or scope boundary, together with its source and supporting premises. Both VLM predictions and conclusions derived by rules use this representation. Scene and task rules derive judgments from the current premises. VLMs supply the remaining semantic judgments. The engine applies the same rules to verify adopted judgments and provide evidence for repair.

3.3.1 Derivation

Rules propagate judgments to consequences or conflicts:

$$K_1 \wedge K_2 \overset{R}{\Longrightarrow} K_3, \qquad K_1 \wedge K_2 \overset{R}{\Longrightarrow} \bot. \tag{5}$$

Here R is an applicable rule and $\bot$ denotes a contradiction. Order judgments imply inequalities between event positions. Spatial qualifiers restrict candidate domains, while scoped avoidance and counts constrain encounters along complete routes. Each derivation records its rule and supporting premises as evidence for subsequent verification and repair. When testing a candidate, the engine treats the other judgments as fixed premises. A conflict excludes the candidate under those premises. If those premises are revised, the engine checks the candidate again.

Global route queries. For a fixed binding β, the solver searches G_{nav} for a complete route satisfying the required visits and orders. It checks scoped avoidance and counts using the entities encountered along the route. A query returns a route that passes replay checks, certifies that no such route exists in the search domain, or remains unresolved. Preference queries additionally seek the minimum violation cost and certify that minimum when sufficient evidence is available. Query evidence depends on the current contract, ASG, and represented search domain (Appendix B.3).

Route requirements determine bindings. The solver tests joint bindings because individually reachable targets may be incompatible under the route constraints. Let $\mathcal{B}_0$ contain bindings allowed

by candidate domains and coreference, and let $\mathcal{F}_\beta$ contain the feasible pairs (π, ω) for binding β:

$$\mathcal{B}_H = \{\beta \in \mathcal{B}_0 : \mathcal{F}_\beta \neq \varnothing\},$$
$$p^*(\beta) = \min_{(\pi,\omega)\in\mathcal{F}_\beta} p(\pi, \omega; P), \qquad \beta \in \mathcal{B}_H. \tag{6}$$

Preferences select the feasible bindings with minimum violation cost:

$$\mathcal{B}_* = \arg\min_{\beta\in\mathcal{B}_H} p^*(\beta). \tag{7}$$

The engine determines a binding when query evidence establishes a unique feasible assignment or a unique preference minimizer. Ties and inconclusive comparisons remain unresolved.

3.3.2 Verification and Evidence-guided Repair

Verification checks adopted judgments using the same rules. A check is marked PASS if its condition is established, BLOCK if it is required and either remains unresolved or detects a conflict, and WARNING otherwise. A check remains unresolved when the available evidence is insufficient to decide whether its condition is satisfied. A conflict between revisable judgments shows that they cannot all hold, but does not identify which judgment is wrong.

For judgments involved in a failed check, repair evidence includes their sources, supporting rules and premises, relevant scene or route facts, and conclusions derived from them. For a demonstrated blocking conflict, the engine withdraws the judgments selected for revision, Δ^-, and their derived conclusions. Rules provide replacements when the remaining judgments uniquely determine them. Otherwise, the VLM proposes replacements using the evidence and u. Let Δ^+ contain the replacement judgments derived by rules or proposed by the VLM. The update is

$$\mathcal{K}' = \left[\mathcal{K} \setminus (\Delta^- \cup \mathrm{Dep}(\Delta^-))\right] \cup \Delta^+, \tag{8}$$

where $\mathrm{Dep}(\Delta^-)$ contains judgments derived directly or indirectly from Δ^-. Affected grounding, binding queries, and routes are recomputed and verified (Appendix B).

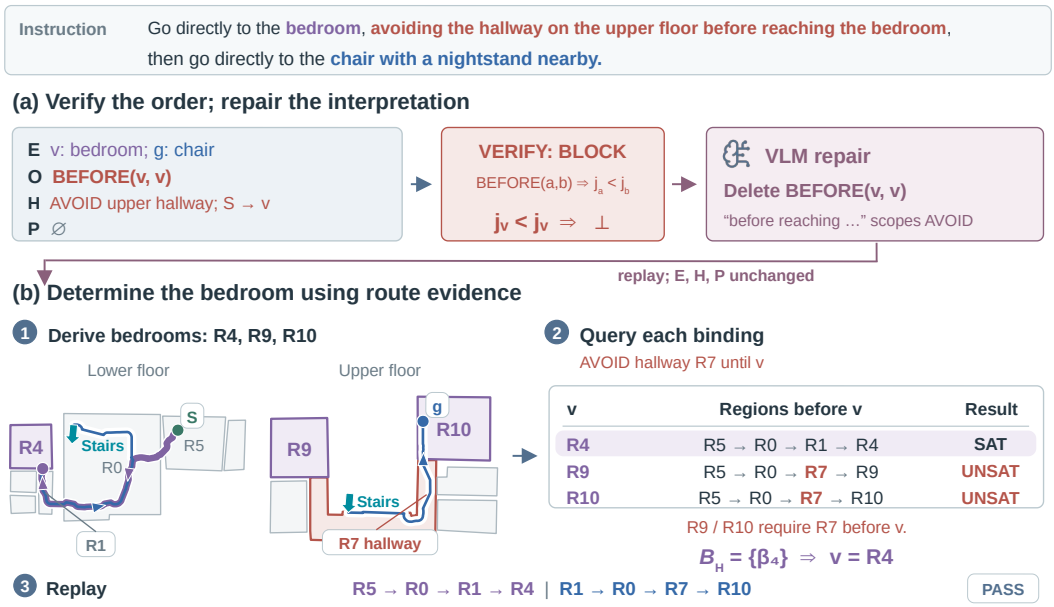


Figure 3: **Determination and repair in one run.** Verification guides VLM repair of an order error, and replayed route queries determine the bedroom target under scoped avoidance. For events a, b, j_a, j_b are waypoint indices, and $\perp$ denotes contradiction. $\mathcal{B}_H$ contains feasible bindings, and β_4 binds v to R4. SAT and UNSAT denote feasibility and infeasibility. R7 is allowed after v.

In Figure 3, verification returns BLOCK for BEFORE(v, v) because it requires $j_v < j_v$. Using the source phrase and this conflict, the VLM removes the incorrect order while preserving scoped

avoidance. The engine then replays the affected grounding and route queries. R9 and R10 are excluded because reaching either crosses the avoided hallway before the bedroom visit, while R4 admits a feasible route. Together, these results determine $\beta(v) = \mathrm{R4}$. The complete route enters the hallway after v, when its avoidance scope has ended. Appendix B.4 gives the full trace.

4 EXPERIMENTS

4.1 BENCHMARK

We construct a benchmark for constrained long-horizon VLN, covering scoped avoidance, distinct-entity counts, and preferences. It evaluates how these constraints affect target selection and whether complete routes satisfy the instruction. Table 1(a) compares its instruction and evaluation focus with existing benchmarks (Anderson et al., 2018; Jain et al., 2019; Ku et al., 2020; Krantz et al., 2020; Qi et al., 2020; Khanna et al., 2024; Song et al., 2025; Yang et al., 2026).

The benchmark contains **192 tasks across nine HM3DSem scenes**, with RGB-D rendered in Habitat (Yadav et al., 2023; Savva et al., 2019). For each task, we specify visits and constraints over scene entities, check that a feasible route exists, and express the task in natural language. Each task includes an exact start pose and annotations used only for evaluating the complete route. No tasks are used for training.

We distinguish tasks by how the target is determined. Hard constraints determine the target in **51 tasks**, and preferences determine it in **38**. In the remaining **103 control tasks**, local qualifiers or direct references identify the target. Table 1(b) shows two independent partitions by task family and how the target is determined. Intermediate visits occur in 55.73% of tasks, constraints scoped to part of the route in 30.21%, and routes spanning multiple floors in 28.65%. Appendix D details benchmark construction.

Table 1: **Benchmark positioning and diversity.** (a) Instruction and evaluation focus. (b) Two independent partitions of the 192 tasks by task family and how the target is determined.

(a) Comparison with existing benchmarks

R2R / R4R / RxR	Route descriptions. Navigation success and path fidelity
VLN-CE	Route following in continuous environments
REVERIE	Object descriptions. Navigation and object grounding
GOAT-Bench	Sequential category, language, or image targets. Success and path efficiency
LHPR-VLN	Sequential navigation subtasks. Subtask success
NavSpace	Spatial instructions. Navigation error and success
Ours	**Visits, scoped avoidance and counts, and preferences. Target selection and complete-route satisfaction**

(b) Task composition and target resolution

Task family	Tasks	Target resolved by	Tasks
Goal grounding	36	Hard constraints	51
Visits / order	36	Preferences	38
Avoidance	21	Relational qualifiers	89
Distinct-entity counts	38	Direct / unique	14
Preferences	34		
Compositions	27		

4.2 EVALUATION SETUP

Baselines. Table 2 compares six VLN methods with their original interfaces and four +ASG adaptations. The adaptations cover NavGPT, GC-VLN, and the LTL planning methods Lang2LTL-2 and LIMP (Liu et al., 2024; Quartey et al., 2025). They receive ground-truth ASGs while retaining their interpretation and planning procedures. *Direct* predicts a grounded contract in the same representation as ReCoNav and uses the shared route solver. *Direct-Feedback* adds validation and solver feedback. Both share ReCoNav's ASG and VLM backbone. General language-model calls

use Qwen3.8-27B-FP8 (Qwen Team, 2026). Models trained for navigation or LTL translation retain their released checkpoints. Appendix D.3 details the methods and adaptations.

Metrics and accounting. Full-instruction satisfaction (**Full IS**) requires a physically valid route satisfying required visits, orders, hard constraints, and all explicit preferences. **Hard NS** omits preference checks. **Prefer** averages the fraction of satisfied preference clauses over tasks containing preferences. ASG methods use all 192 tasks for Full IS and Hard NS. The original interfaces cannot initialize 14 prescribed start poses, so these rates use the remaining 178 tasks. Prefer uses 54 tasks for ASG methods and 51 for the original interfaces. Budget failures and runtime exceptions score zero within these fixed populations. Run completion is measured over all 192 issued tasks. Token and time means use completed runs. Appendix D.2 details accounting.

4.3 Main Results

ReCoNav achieves **81.25% Full IS**, compared with 24.48% for NavGPT+ASG, the strongest evaluated existing method (Table 2). With the same ASG, VLM backbone, and route solver, Direct-Feedback reaches 44.27%, using 113.15k tokens per completed run versus ReCoNav's **7.12k**.

Table 2: **Navigation and reasoning baselines.** Rates (%). Full IS and Hard NS use 178 original-interface or 192 ASG tasks (Prefer: 51 or 54, respectively). Run comp. is run completion over all 192 tasks. Cost means use completed runs. n/a: uninstrumented.

Method	Full IS↑	Hard NS↑	Prefer↑	Run comp.↑	Tokens (k)↓	Time (s)↓
Original interfaces						
NavGPT	6.74	8.43	38.24	92.19	42.12	580.4
MapGPT	8.43	10.11	44.12	91.67	29.84	163.0
CA-Nav	0.56	1.69	33.33	92.71	1.27	199.7
GC-VLN	0.00	0.00	41.18	92.71	0.35	393.3
SNav	2.81	3.37	36.27	92.71	n/a	56.6
DualVLN	1.69	2.81	39.22	92.71	n/a	87.8
With ground-truth ASG						
NavGPT+ASG	24.48	26.56	54.63	99.48	418.41	806.7
GC-VLN+ASG	3.13	3.13	39.81	100.00	0.35	120.7
Lang2LTL-2+ASG	7.81	7.81	50.00	100.00	2.22	26.6
LIMP+ASG	5.21	7.81	38.89	98.96	1.09	7.0
Shared ASG and constraint solver						
Direct	27.08	28.13	22.22	99.48	47.98	118.5
Direct-Feedback	44.27	48.44	38.89	98.96	113.15	282.4
ReCoNav	**81.25**	**85.42**	**79.63**	99.48	7.12	100.1

4.4 Global Context and Constraint Reasoning

To examine how global scene information and constraint reasoning affect target selection, we analyze tasks where hard constraints determine the target. Of these 51 tasks, 44 have start poses supported by all methods. We refer to this subset as Hard44.

With ASG access, NavGPT rises from 15.91% to 38.64% Full IS and GC-VLN from 0% to 6.82%. ReCoNav reaches **84.09% Full IS** (37/44), exceeding all four +ASG adaptations, including the LTL planning methods Lang2LTL-2 and LIMP. Direct and Direct-Feedback share ReCoNav's ASG, contract representation, route solver, and Qwen VLM backbone, and reach 13.64% and 29.55%, respectively.

Together, Tables 2 and 3 suggest that global scene information can improve instruction satisfaction. The comparisons with the +ASG methods, Direct, and Direct-Feedback indicate that performance also depends on how constraints are used in instruction interpretation, target selection, and route planning. Hard44 further shows that this pattern holds when hard constraints determine which candidate target to visit.

Table 3: **Hard-constraint-driven target resolution (Hard44).** Rates (%) use the same 44 tasks for all methods (Prefer: four). Failures score zero. Run comp. is run completion. Cost means use completed runs.

Method	Full IS↑	Hard NS↑	Prefer↑	Run comp.↑	Tokens (k)↓	Time (s)↓
NavGPT	15.91	15.91	25.00	100.00	38.51	534.6
MapGPT	11.36	11.36	50.00	100.00	27.60	154.4
CA-Nav	0.00	0.00	50.00	100.00	1.26	198.6
GC-VLN	0.00	0.00	50.00	100.00	0.34	434.8
SNav	4.55	4.55	50.00	100.00	n/a	52.6
DualVLN	4.55	4.55	50.00	100.00	n/a	86.1
NavGPT+ASG	38.64	38.64	50.00	100.00	394.50	782.3
GC-VLN+ASG	6.82	6.82	25.00	100.00	0.34	108.5
Lang2LTL-2+ASG	2.27	2.27	25.00	100.00	2.21	25.6
LIMP+ASG	9.09	9.09	50.00	100.00	1.08	6.3
Direct	13.64	13.64	0.00	100.00	47.23	116.3
Direct-Feedback	29.55	29.55	50.00	100.00	124.74	330.1
ReCoNav	**84.09**	**84.09**	**100.00**	100.00	7.30	99.7

4.5 Mechanism Ablations

We use the same 192 tasks and GT-ASGs with GPT-5.6 Sol (OpenAI, 2026), Gemini 3.1 Pro Preview (Google DeepMind, 2026a), Qwen3.8-27B (Qwen Team, 2026), and Gemma4-31B (Gemma Team, 2026). Across all four backbones, ReCoNav improves Full IS with fewer tokens than Direct and Direct-Feedback (Table 4). Appendix D.4 reports the remaining metrics.

Table 4: **Task modeling and engine ablations.** Full IS (%) over all 192 tasks and mean tokens (k) over completed runs.

	GPT		Gemini		Qwen		Gemma	
Variant	Full IS↑	Tokens↓	Full IS↑	Tokens↓	Full IS↑	Tokens↓	Full IS↑	Tokens↓
Direct	91.15	40.32	26.56	63.88	27.08	47.98	27.08	48.52
Direct-Feedback	96.35	44.01	58.33	149.21	44.27	113.15	49.48	110.02
Direct-ID-Sequence	86.46	31.13	81.25	45.64	42.71	41.97	53.13	42.87
Forced-Early	68.23	16.05	67.19	38.32	56.25	10.09	64.06	10.04
No Verify/Repair	98.96	**9.19**	96.35	**16.05**	73.44	**5.69**	93.23	**6.36**
ReCoNav	**100.00**	9.27	**98.44**	17.08	**81.25**	7.12	**95.83**	6.67

Instruction contracts and reasoning. *Direct-ID-Sequence* predicts target entity IDs in visit order. The solver connects these targets without enforcing the instruction's avoidance, counting, or preference clauses. It outperforms Direct on three backbones, including 42.71% versus 27.08% Full IS with Qwen. Predicting a contract alone therefore does not ensure better performance. ReCoNav uses the contract for derivation, verification, and repair, outperforming both variants across all four backbones (81.25% Full IS with Qwen).

Global propagation for target selection. *Forced-Early* selects targets from semantic and spatial matches before the solver checks route constraints. It retains repair and final route checks. ReCoNav achieves higher Full IS than Forced-Early across all four backbones, reaching **100% versus 68.23%** with GPT and **81.25% versus 56.25%** with Qwen (Table 4). These results support using complete-route feasibility and preference costs to guide target selection. Appendix D.4 provides the subgroup analysis.

Verification with repair. *No Verify/Repair* omits the additional verification and repair stage while retaining derivation, joint target binding, and route-feasibility checks. Restoring this stage improves Full IS across all four backbones. It raises GPT from 98.96% to 100% and Qwen from **73.44% to 81.25%**, with Qwen tokens increasing from 5.69k to 7.12k.

4.6 Real-Scene Visualization

We construct an ASG from a scan of a real indoor scene, using predicted object labels and manually annotated regions and portals. The avoidance constraint excludes the bin inside the office. ReCoNav therefore selects the other bin and plans a 14.53 m route that visits it before the washing machine (Figure 4).

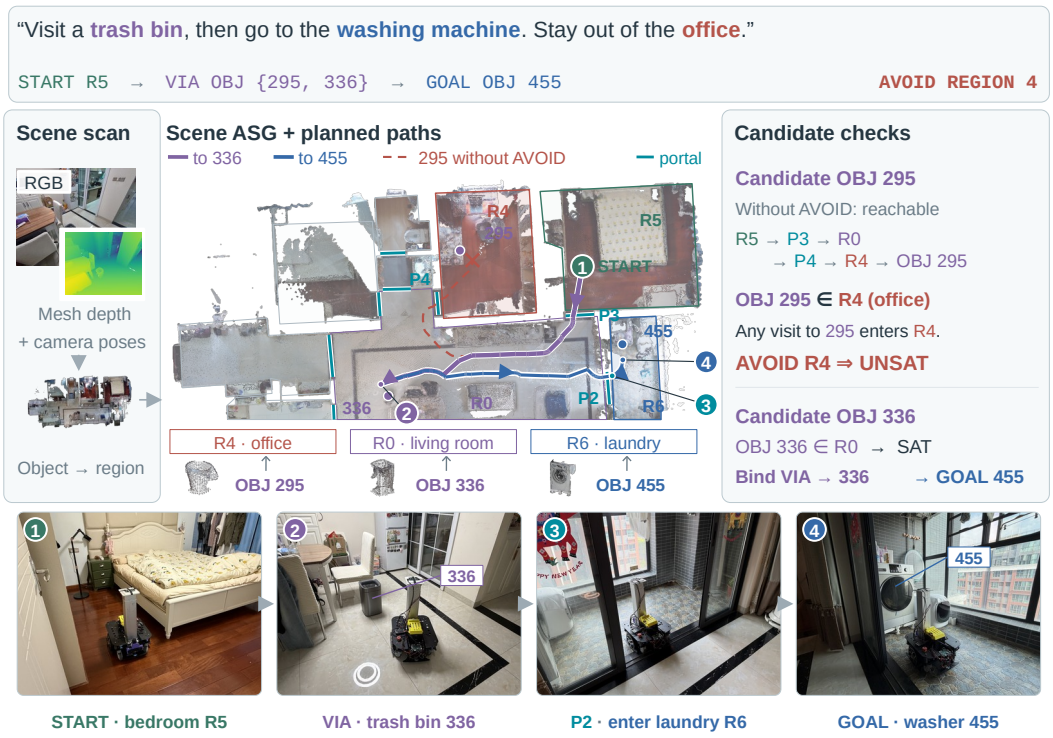


Figure 4: **Real-Scene Visualization.** Scene scan, ASG, candidate checks, and planned route. Robot photographs correspond to the four numbered route locations: the start, selected bin, laundry doorway, and washing machine. P denotes a portal, and OBJ denotes an object.

5 Limitations and Future Work

ReCoNav verifies judgments relative to the extracted instruction contract and scene representation. A semantically incorrect but internally consistent contract may escape verification (Ye et al., 2023). Reliable navigation also depends on the ASG and navigation graph accurately representing scene entities and traversability. Object and room recognition errors can affect navigation (Werby et al., 2024). Our quantitative evaluation of ReCoNav uses fixed, ground-truth ASGs to isolate task reasoning. A next step is to integrate online 3D semantic mapping (Deng et al., 2026; Zhai et al., 2026) and incremental construction of scene graphs (Hughes et al., 2022), updating affected judgments and routes as the ASG changes.

6 Conclusion

We introduce a benchmark of 192 tasks across nine indoor scenes for constrained long-horizon VLN. ReCoNav combines an instruction contract, an ASG, and a Determination Engine to propagate constraints across instruction interpretation, scene grounding, and complete-route planning. The engine applies scene and task rules to derive and verify judgments and guide VLM repair. ReCoNav achieves the highest full-instruction satisfaction among the evaluated methods, while maintaining low mean token usage per completed run across four VLM backbones.

AI USE STATEMENT

Generative AI assisted writing, literature search, coding and auditing, baseline adaptation, experiments and analysis, and figure preparation. Gemini-3.7-Flash and GPT-5.6-Sol generated benchmark instructions from author-defined symbolic tasks. The authors developed the research question, core method, formalization, and benchmark design, reviewed AI-assisted outputs, and take responsibility for the final text, code, data, and results. Language and vision-language models are also components of the evaluated systems.

REPRODUCIBILITY STATEMENT

Section 3 and Appendices A–D document the formulation, reasoning and search procedures, scene construction, evaluation protocols and settings, and complete ablation results. We plan to release a cleaned implementation and benchmark.

REFERENCES

Peter Anderson, Qi Wu, Damien Teney, Jake Bruce, Mark Johnson, Niko Sünderhauf, Ian D. Reid, Stephen Gould, and Anton van den Hengel. Vision-and-Language Navigation: Interpreting Visually-Grounded Navigation Instructions in Real Environments. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*, pp. 3674–3683. Computer Vision Foundation / IEEE Computer Society, 2018. doi: 10.1109/CVPR.2018.00387. URL http://openaccess.thecvf.com/content_cvpr_2018/html/Anderson_Vision-and-Language_Navigation_Interpreting_CVPR_2018_paper.html.

Neel P. Bhatt, Yunhao Yang, Rohan Siva, Pranay Samineni, Daniel Milan, Zhangyang Wang, and Ufuk Topcu. VLN-Zero: Rapid exploration and cache-enabled neurosymbolic vision-language planning for zero-shot transfer in robot navigation. In *3rd International Conference on Neuro-Symbolic Systems (NeuS)*, 2026.

Jiaqi Chen, Bingqian Lin, Ran Xu, Zhenhua Chai, Xiaodan Liang, and Kwan-Yee Kenneth Wong. MapGPT: Map-Guided Prompting with Adaptive Path Planning for Vision-and-Language Navigation. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pp. 9796–9810. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.529. URL https://doi.org/10.18653/v1/2024.acl-long.529.

Kehan Chen, Dong An, Yan Huang, Rongtao Xu, Yifei Su, Yonggen Ling, Ian D. Reid, and Liang Wang. Constraint-Aware Zero-Shot Vision-Language Navigation in Continuous Environments. *IEEE Trans. Pattern Anal. Mach. Intell.*, 47(11):10441–10456, 2025. doi: 10.1109/TPAMI.2025.3594204.

Shizhe Chen, Pierre-Louis Guhur, Makarand Tapaswi, Cordelia Schmid, and Ivan Laptev. Think Global, Act Local: Dual-scale Graph Transformer for Vision-and-Language Navigation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*, pp. 16516–16526. IEEE, 2022. doi: 10.1109/CVPR52688.2022.01604.

Giuseppe De Giacomo and Moshe Y. Vardi. Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. In Francesca Rossi (ed.), *IJCAI 2013, Proceedings of the 23rd International Joint Conference on Artificial Intelligence, Beijing, China, August 3-9, 2013*, pp. 854–860. IJCAI/AAAI, 2013. URL http://www.aaai.org/ocs/index.php/IJCAI/IJCAI13/paper/view/6997.

Leonardo Mendonça de Moura and Nikolaj S. Bjørner. Z3: An Efficient SMT Solver. In C. R. Ramakrishnan and Jakob Rehof (eds.), *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings*, volume 4963 of *Lecture Notes in Computer Science*, pp. 337–340. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24. URL https://doi.org/10.1007/978-3-540-78800-3_24.

Zilong Deng, Federico Tombari, Marc Pollefeys, Johanna Wald, and Daniel Barath. OVI-MAP: Open-vocabulary instance-semantic mapping. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12606–12616, June 2026.

Jon Doyle. A Truth Maintenance System. *Artif. Intell.*, 12(3):231–272, 1979. doi: 10.1016/0004-3702(79)90008-0.

Gemma Team. Gemma 4 technical report, 2026. URL https://arxiv.org/abs/2607.02770.

Google DeepMind. Gemini 3.1 Pro, 2026a. URL https://deepmind.google/models/model-cards/gemini-3-1-pro/.

Google DeepMind. Gemini 3.7 Flash, 2026b. URL https://deepmind.google/models/model-cards/gemini-3-7-flash/.

Qiao Gu, Ali Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, Bipasha Sen, Aditya Agarwal, Corban Rivera, William Paul, Kirsty Ellis, Rama Chellappa, Chuang Gan, Celso Miguel de Melo, Joshua B. Tenenbaum, Antonio Torralba, Florian Shkurti, and Liam Paull. ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning. In *IEEE International Conference on Robotics and Automation, ICRA 2024, Yokohama, Japan, May 13-17, 2024*, pp. 5021–5028. IEEE, 2024. doi: 10.1109/ICRA57147.2024.10610243.

Zebin Han, Xudong Wang, Baichen Liu, Qi Lyu, Zhenduo Shang, Jiahua Dong, Lianqing Liu, and Zhi Han. SeqWalker: Sequential-Horizon Vision-and-Language Navigation with Hierarchical Planning. In Sven Koenig, Chad Jenkins, and Matthew E. Taylor (eds.), *Fortieth AAAI Conference on Artificial Intelligence, Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence, Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2026, Singapore, January 20-27, 2026*, pp. 18279–18287. AAAI Press, 2026. doi: 10.1609/AAAI.V40I22.38891.

Yilun Hao, Yang Zhang, and Chuchu Fan. Planning Anything with Rigor: General-Purpose Zero-Shot Planning with LLM-based Formalized Programming. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=0K1OaL6XuK.

Chenguang Huang, Oier Mees, Andy Zeng, and Wolfram Burgard. Visual Language Maps for Robot Navigation. In *IEEE International Conference on Robotics and Automation, ICRA 2023, London, UK, May 29 - June 2, 2023*, pp. 10608–10615. IEEE, 2023. doi: 10.1109/ICRA48891.2023.10160969.

Sheng-Yu Huang, Jaesung Choe, Yu-Chiang Frank Wang, and Cheng Sun. OpenVoxel: Training-free grouping and captioning voxels for open-vocabulary 3D scene understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16734–16745, 2026.

Nathan Hughes, Yun Chang, and Luca Carlone. Hydra: A Real-time Spatial Perception System for 3D Scene Graph Construction and Optimization. In Kris Hauser, Dylan A. Shell, and Shoudong Huang (eds.), *Robotics: Science and Systems XVIII, New York City, NY, USA, June 27 - July 1, 2022*, 2022. doi: 10.15607/RSS.2022.XVIII.050.

Vihan Jain, Gabriel Magalhães, Alexander Ku, Ashish Vaswani, Eugene Ie, and Jason Baldridge. Stay on the Path: Instruction Fidelity in Vision-and-Language Navigation. In Anna Korhonen, David R. Traum, and Lluís Màrquez (eds.), *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, pp. 1862–1872. Association for Computational Linguistics, 2019. doi: 10.18653/V1/P19-1181.

Mukul Khanna, Ram Ramrakhya, Gunjan Chhablani, Sriram Yenamandra, Théophile Gervet, Matthew Chang, Zsolt Kira, Devendra Singh Chaplot, Dhruv Batra, and Roozbeh Mottaghi. GOAT-Bench: A Benchmark for Multi-Modal Lifelong Navigation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*, pp. 16373–16383. IEEE, 2024. doi: 10.1109/CVPR52733.2024.01549.

Jacob Krantz, Erik Wijmans, Arjun Majumdar, Dhruv Batra, and Stefan Lee. Beyond the Nav-Graph: Vision-and-Language Navigation in Continuous Environments. In Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm (eds.), *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part XXVIII*, volume 12373 of *Lecture Notes in Computer Science*, pp. 104–120. Springer, 2020. doi: 10.1007/978-3-030-58604-1_7. URL https://doi.org/10.1007/978-3-030-58604-1_7.

Alexander Ku, Peter Anderson, Roma Patel, Eugene Ie, and Jason Baldridge. Room-Across-Room: Multilingual Vision-and-Language Navigation with Dense Spatiotemporal Grounding. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online,*

November 16-20, 2020, pp. 4392–4412. Association for Computational Linguistics, 2020. doi: 10.18653/V1/2020.EMNLP-MAIN.356. URL https://doi.org/10.18653/v1/2020.emnlp-main.356.

Kailing Li, Tianwen Qian, Lijin Yang, Yuqian Fu, Jingyu Gong, Xiaoling Wang, and Liang He. Bridging the 2D-3D gap: A hierarchical semantic-geometric map for vision language navigation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15243–15252, 2026.

Jason Xinyu Liu, Ankit Shah, George Konidaris, Stefanie Tellex, and David Paulius. Lang2LTL-2: Grounding Spatiotemporal Navigation Commands Using Large Language and Vision-Language Models. In *IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2024, Abu Dhabi, United Arab Emirates, October 14-18, 2024*, pp. 2325–2332. IEEE, 2024. doi: 10.1109/IROS58592.2024.10802696.

OpenAI. GPT-5.6 Sol, 2026. URL https://developers.openai.com/api/docs/models/gpt-5.6-sol.

Yuankai Qi, Qi Wu, Peter Anderson, Xin Wang, William Yang Wang, Chunhua Shen, and Anton van den Hengel. REVERIE: Remote Embodied Visual Referring Expression in Real Indoor Environments. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*, pp. 9979–9988. Computer Vision Foundation / IEEE, 2020. doi: 10.1109/CVPR42600.2020.01000. URL https://openaccess.thecvf.com/content_CVPR_2020/html/Qi_REVERIE_Remote_Embodied_Visual_Referring_Expression_in_Real_Indoor_Environments_CVPR_2020_paper.html.

Benedict Quartey, Eric Rosen, Stefanie Tellex, and George Konidaris. Verifiably Following Complex Robot Instructions with Foundation Models. In *IEEE International Conference on Robotics and Automation, ICRA 2025, Atlanta, GA, USA, May 19-23, 2025*, pp. 1–8. IEEE, 2025. doi: 10.1109/ICRA55743.2025.11127418.

Qwen Team. Qwen3.8-27B-FP8, 2026. URL https://huggingface.co/Qwen/Qwen3.8-27B-FP8.

Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Suenderhauf. SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning. In Jie Tan, Marc Toussaint, and Kourosh Darvish (eds.), *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pp. 23–72. PMLR, 2023. URL https://proceedings.mlr.press/v229/rana23a.html.

Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloé Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross B. Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment Anything in Images and Videos. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=Ha6RTeWMd0.

Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, Devi Parikh, and Dhruv Batra. Habitat: A Platform for Embodied AI Research. In *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*, pp. 9338–9346. IEEE, 2019. doi: 10.1109/ICCV.2019.00943.

Xinshuai Song, Weixing Chen, Yang Liu, Weikai Chen, Guanbin Li, and Liang Lin. Towards Long-Horizon Vision-Language Navigation: Platform, Benchmark and Method. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2025, Nashville, TN, USA, June 11-15, 2025*, pp. 12078–12088. Computer Vision Foundation / IEEE, 2025. doi: 10.1109/CVPR52734.2025.01128. URL https://openaccess.thecvf.com/content/CVPR2025/html/Song_Towards_Long-Horizon_Vision-Language_Navigation_Platform_Benchmark_and_Method_CVPR_2025_paper.html.

Meng Wei, Chenyang Wan, Jiaqi Peng, Xiqian Yu, Yuqiang Yang, Delin Feng, Wenzhe Cai, Chenming Zhu, Tai Wang, Jiangmiao Pang, and Xihui Liu. Ground Slow, Move Fast: A Dual-System Foundation Model for Generalizable Vision-Language Navigation. In Carl Vondrick, Bharath Hariharan, Colin Raffel, Lerrel Pinto, Diyi Yang, and Aleksandra Faust (eds.), *The Fourteenth International Conference on Learning Representations, ICLR 2026, Rio de Janeiro, Brazil, April 23-27, 2026*. proceedings.iclr.cc, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/14da7aea05debb963b3d8d46449d51a0-Abstract-Conference.html.

Abdelrhman Werby, Chenguang Huang, Martin Büchner, Abhinav Valada, and Wolfram Burgard. Hierarchical Open-Vocabulary 3D Scene Graphs for Language-Grounded Robot Navigation. In Dana Kulic, Gentiane Venture, Kostas E. Bekris, and Enrique Coronado (eds.), *Robotics: Science and Systems XX, Delft, The Netherlands, July 15-19, 2024*, 2024. doi: 10.15607/RSS.2024.XX.077.

Karmesh Yadav, Ram Ramrakhya, Santhosh Kumar Ramakrishnan, Théophile Gervet, John M. Turner, Aaron Gokaslan, Noah Maestre, Angel Xuan Chang, Dhruv Batra, Manolis Savva, Alexander William Clegg, and Devendra Singh Chaplot. Habitat-Matterport 3D Semantics Dataset. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*, pp. 4927–4936. IEEE, 2023. doi: 10.1109/CVPR52729.2023.00477.

Mi Yan, Jiazhao Zhang, Yan Zhu, and He Wang. MaskClustering: View Consensus Based Mask Graph Clustering for Open-Vocabulary 3D Instance Segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*, pp. 28274–28284. IEEE, 2024. doi: 10.1109/CVPR52733.2024.02671.

Haolin Yang, Yuxing Long, Zhuoyuan Yu, Zihan Yang, Minghan Wang, Jiapeng Xu, Yihan Wang, Ziyan Yu, Wenzhe Cai, Lei Kang, and Hao Dong. NavSpace: How navigation agents follow spatial intelligence instructions, 2026. URL https://arxiv.org/abs/2510.08173.

Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. SatLM: Satisfiability-Aided Language Models Using Declarative Prompting. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.

Hang Yin, Haoyu Wei, Xiuwei Xu, Wenxuan Guo, Jie Zhou, and Jiwen Lu. GC-VLN: Instruction as graph constraints for training-free vision-and-language navigation. In Joseph Lim, Shuran Song, and Hae-Won Park (eds.), *Proceedings of the 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pp. 1809–1824. PMLR, 2025. URL https://proceedings.mlr.press/v305/yin25a.html.

Hongjia Zhai, Qi Zhang, Xiaokun Pan, Xiyu Zhang, Yitong Dong, Huaqi Zhang, Dan Xu, and Guofeng Zhang. OnlinePG: Online open-vocabulary panoptic mapping with 3D gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 33269–33279, June 2026.

Jiazhao Zhang, Kunyu Wang, Rongtao Xu, Gengze Zhou, Yicong Hong, Xiaomeng Fang, Qi Wu, Zhizheng Zhang, and He Wang. NaVid: Video-based VLM Plans the Next Step for Vision-and-Language Navigation. In Dana Kulic, Gentiane Venture, Kostas E. Bekris, and Enrique Coronado (eds.), *Robotics: Science and Systems XX, Delft, The Netherlands, July 15-19, 2024*, 2024. doi: 10.15607/RSS.2024.XX.079.

Jiwen Zhang, Zejun Li, Siyuan Wang, Xiangyu Shi, Zhongyu Wei, and Qi Wu. SpatialNav: Leveraging Spatial Scene Graphs for Zero-Shot Vision-and-Language Navigation. *CoRR*, abs/2601.06806, 2026. doi: 10.48550/ARXIV.2601.06806.

Lingfeng Zhang, Xiaoshuai Hao, Qinwen Xu, Qiang Zhang, Xinyao Zhang, Pengwei Wang, Jing Zhang, Zhongyuan Wang, Shanghang Zhang, and Renjing Xu. MapNav: A Novel Memory Representation via Annotated Semantic Maps for VLM-based Vision-and-Language Navigation. In

Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pp. 13032–13056. Association for Computational Linguistics, 2025. doi: 10.18653/V1/2025.ACL-LONG.638.

Gengze Zhou, Yicong Hong, and Qi Wu. NavGPT: Explicit Reasoning in Vision-and-Language Navigation with Large Language Models. In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan (eds.), *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20-27, 2024, Vancouver, Canada*, pp. 7641–7649. AAAI Press, 2024. doi: 10.1609/AAAI.V38I7.28597. URL https://doi.org/10.1609/aaai.v38i7.28597.

Ziyu Zhu, Xilin Wang, Yixuan Li, Zhuofan Zhang, Xiaojian Ma, Yixin Chen, Baoxiong Jia, Wei Liang, Qian Yu, Zhidong Deng, Siyuan Huang, and Qing Li. Move to Understand a 3D Scene: Bridging Visual Grounding and Exploration for Efficient and Versatile Embodied Navigation. In *IEEE/CVF International Conference on Computer Vision, ICCV 2025, Honolulu, HI, USA, October 19-25, 2025*, pp. 8120–8132. IEEE, 2025. doi: 10.1109/ICCV51701.2025.00761.

A CONTRACT SEMANTICS

The contract's visits, order, scoped avoidance and counts, and preferences are all *route constraints*. They share one target assignment and event realization. Their consequences can therefore constrain interpretation and grounding as well as the final route.

Waypoint occurrences. The waypoint sequence includes the selected visit occurrences. A connecting motion may be split at an occurrence without changing the physical path. Each realized event has its own waypoint index j_i; successive events may share a physical pose and are then represented by separate entries. START has index 0 and GOAL has index n.

Descriptions and realizations. Descriptions are fields of events and constraint clauses in $\mathcal{U} = (E, O, H, P)$. Each retains a nominal phrase, spatial qualifiers, and any coreference relation. Event grounding selects $\beta(e_i) \in N_i \subseteq N$; AVOID and COUNT descriptions denote sets of matching entities. Spatial qualifiers in a description are conjunctive and apply to the selected target. For a realized event, $\mathrm{IN}(e_i, n_y)$ requires $\mathrm{in}(\beta(e_i), n_y)$. A qualifier *near* object o' requires $\mathrm{near}(\beta(e_i), o')$. The start s_0 anchors start-relative distance comparisons. If a set description uses a resolved event target as a spatial reference, its denotation uses that same target. A realization ω records these consistent target choices and one selected occurrence for each realized event. An optional preferred visit may have no selected occurrence. Two visits to the same entity remain distinct events and may be ordered independently. All predicates use this shared realization.

A.1 SPATIAL RELATIONS

Membership and portal relations. For an object o, region r, level ℓ, and portal p, membership and incidence are defined using the relations in M:

$$\begin{aligned}
\mathrm{in}(o, r) &\iff (o, r) \in M, \\
\mathrm{in}(r, \ell) &\iff (r, \ell) \in M, \\
\mathrm{in}(o, \ell) &\iff \exists r : \mathrm{in}(o, r) \wedge \mathrm{in}(r, \ell), \\
\mathrm{in}(p, r) &\iff (p, r) \in M.
\end{aligned}$$

For scene entities $n_x, n_y \in N$, we write $n_x \in n_y$ for $\mathrm{in}(n_x, n_y)$ when n_y is a region or level. Same-kind filters use $\mathrm{in}(r, r') \iff r = r'$ and $\mathrm{in}(\ell, \ell') \iff \ell = \ell'$. For a reference set of regions, objects and regions match any member; a portal must be incident to every listed region. Level references match membership in any listed level.

The spatial qualifier for a portal between distinct regions is

$$\mathrm{between}(p, r_1, r_2) \iff r_1 \neq r_2 \ \wedge\ \mathrm{in}(p, r_1) \wedge \mathrm{in}(p, r_2).$$

Reference sets extend this definition by matching one region from each set. Two regions are connected when such a portal exists. This qualifier describes a portal target; event-order BETWEEN is defined below.

Near. Let $d_\Gamma(o, o')$ be the minimum Euclidean distance between the horizontal object footprints represented in Γ. For objects o, o',

$$\begin{aligned}
\mathrm{near}(o, o') \iff\ & o \neq o' \ \wedge\ d_\Gamma(o, o') \leq 2\,\mathrm{m} \\
& \wedge\ \exists r : \mathrm{in}(o, r) \wedge \mathrm{in}(o', r).
\end{aligned}$$

A singular reference resolves to one object. An explicitly existential class reference is satisfied when this predicate holds for at least one matching object.

Nearest and farthest. Let C be the object candidates after nominal, membership, and near matching, restricted to the reference object's level and excluding the reference object itself. Then

$$\begin{aligned}
\mathrm{nearest}(o, o'; C) &\iff o \in \arg\min_{\tilde{o} \in C} d_\Gamma(\tilde{o}, o'), \\
\mathrm{farthest}(o, o'; C) &\iff o \in \arg\max_{\tilde{o} \in C} d_\Gamma(\tilde{o}, o').
\end{aligned}$$

For a START reference, replace o' by the original task start s_0, use its horizontal point for distance, and compare on its containing level. An empty comparison domain selects no target. All tied extrema remain candidates; distance comparisons use a 10^{-6} m tolerance. Multiple extreme qualifiers use the same candidates left by the preceding filters, with each reference determining its comparison level. These qualifiers restrict N_i before route-feasibility queries. AVOID, COUNT, and preferred target descriptions use the same predicates.

A.2 Route Constraints

Table 5: Contract operators. Ordering uses requested event witnesses; scoped conditions use the physical encounter trace.

Operator	Semantics
VIA / GOAL	A visit witness / a terminal visit witness for the target.
BEFORE / AFTER	Strict order between two requested occurrences.
NEXT	Consecutive requested milestone witnesses, allowing physical motion between them.
BETWEEN–ONLY	Exactly the named milestones occur between the boundary witnesses.
BETWEEN–WITHOUT	The named milestones may not occur between the boundary witnesses.
AVOID	No target-entity encounter strictly within the scope.
COUNT	The number of distinct target identities in the scope satisfies $\leq$, $=$, or $\geq k$.
PREFER	A visit, order, avoidance, or count predicate incurs its violation cost if unsatisfied.

Order predicates. Let $E_\omega \subseteq E$ be the realized requested events. For realized boundary witnesses with $j_a < j_b$, define $B_{ab}(\omega) = \{e_i \in E_\omega : j_a < j_i < j_b\}$. BEFORE$(e_a, e_b)$ and AFTER(e_b, e_a) both require $j_a < j_b$. For a named event set $F \subseteq E$, the remaining predicates require

$$\begin{aligned} \text{NEXT}(e_a, e_b): &\quad j_a < j_b \ \wedge \ B_{ab}(\omega) = \varnothing, \\ \text{BETWEEN-ONLY}(e_a, e_b, F): &\quad j_a < j_b \ \wedge \ B_{ab}(\omega) = F, \\ \text{BETWEEN-WITHOUT}(e_a, e_b, F): &\quad j_a < j_b \ \wedge \ B_{ab}(\omega) \cap F = \varnothing. \end{aligned}$$

These predicates concern requested event witnesses; incidental physical encounters are evaluated by the scoped constraints.

Encounter trace. Let $\tau(\pi) = ((v_\ell, \xi_\ell))_{\ell=1}^{L}$ be the ordered physical encounters, where $v_\ell \in N$ and $\xi_\ell \in [0, n]$ records an encounter's position along the route. Waypoint q_j has position j; positions strictly between j and $j+1$ lie on their connecting motion. For $\sigma = (e_a, e_b)$,

$$N_\sigma(\pi, \omega) = \{v_\ell : j_a < \xi_\ell < j_b\}.$$

The geometry of each entity determines its encounter condition:

Entity	Visit geometry $D(v)$	Encounter recorded in $\tau(\pi)$
Object	Approach neighborhood of its footprint	Entry into that neighborhood
Region	Navigable region interior	Entry from outside the region
Portal	Registered passage interface	Crossing between its two sides
Level	Navigable extent of its regions	Transition into the level

Encounters are computed along complete waypoint segments, so unrequested objects still affect AVOID and COUNT. Initial occupancy does not create a new entry. Leaving and re-entering creates another occurrence but does not increase a distinct-identity count. Only the encounter at an actual scope boundary is excluded; an earlier encounter with the eventual goal remains relevant. A local planning-leg endpoint does not create a new scope boundary. The boundaries must have selected occurrences with $j_a < j_b$. Omitted boundaries default to START and GOAL.

Conditional orders and preferences. Orders involving optional preferred visits apply when all their referenced events are realized. An unselected preferred visit incurs its violation cost. Each preferred clause incurs its cost at most once, regardless of repeated violations. The minimization

in Equation 4 uses one ω for all clauses; it cannot select different target identities or occurrences to satisfy individual clauses independently. Distinct-entity counts over neighboring scopes cannot generally be added, because their identity sets can overlap.

B Determination Engine

The Determination Engine applies the same scene and task relations to derive decisions, verify adopted judgments, and provide evidence for repair. Target selection is one application of this shared mechanism.

B.1 Derivation

Let $\mathcal{K}$ contain the current judgments and $\mathcal{R}$ the applicable rules. A VLM prediction enters $\mathcal{K}$ as a revisable premise. A rule application records its premises, rule, and conclusion, using either form in Equation 5: a new judgment or a conflict $\bot$.

Local rules combine admissible entity kinds, semantic matches, membership, and geometry to restrict candidate domains. Route queries then propagate the contract's requirements across interpretation, grounding, and planning. This propagation preserves each clause's scope: a requirement restricted to one route portion can affect an upstream decision without becoming a whole-mission restriction. Table 6 summarizes the core rules and the evidence recorded for their application.

Candidate exclusion. For a candidate hypothesis K_1, a conflict $K_1 \wedge K_2 \overset{R}{\Longrightarrow} \bot$ excludes it under the premises K_2 held fixed for that query.

Incomplete route evidence. For the joint binding domain $\mathcal{B}_0$, let $\mathcal{B}_{\mathrm{sat}}$ contain assignments with verified feasible witnesses, $\mathcal{B}_{\mathrm{unsat}}$ those with infeasibility certificates, and $\mathcal{B}_?$ the remaining assignments. These sets partition $\mathcal{B}_0$, and valid query evidence gives

$$\mathcal{B}_{\mathrm{sat}} \subseteq \mathcal{B}_H \subseteq \mathcal{B}_{\mathrm{sat}} \cup \mathcal{B}_?. \tag{9}$$

One feasible assignment determines the binding only after all competitors have been excluded. An inconclusive feasibility query leaves its assignment in $\mathcal{B}_?$.

If several assignments are feasible, their certified preference minima define $\mathcal{B}_*$ in Equation 7. A unique minimum determines a binding; equal minima preserve alternatives. Path length and entity ID do not resolve this semantic ambiguity.

B.2 Verification and Evidence-guided Repair

Verification. A check a applies a rule relation to adopted judgments. Its result $v_a = \mathrm{Verify}_a(\mathcal{K}) \in \{\top, \bot, ?\}$ denotes an established obligation, a demonstrated violation, or insufficient evidence. Let $b_a = 1$ when establishing this obligation is required at the current stage:

$$\mathrm{status}(a) = \begin{cases} \text{PASS}, & v_a = \top, \\ \text{BLOCK}, & v_a \neq \top \ \wedge \ b_a = 1, \\ \text{WARNING}, & \text{otherwise.} \end{cases} \tag{10}$$

An unknown required check blocks acceptance without proving a contradiction.

Suppose K_1 and K_2 are both revisable judgments and $K_1 \wedge K_2 \overset{R}{\Longrightarrow} \bot$. The rule establishes that they cannot both hold; it does not establish which judgment is wrong. For example, extracted orders $e_a \prec e_b$ and $e_b \prec e_a$ imply $j_a < j_b < j_a$, but either order may have been misinterpreted. Likewise, requiring at least three distinct entities from a grounded set of size two exposes an inconsistency whose source may be the requested number or the grounding. These illustrative checks do not by themselves determine a semantic replacement.

Table 6: **Core scene and task rules.** Rules derive judgments, check adopted judgments, and provide evidence for repair. Global binding conclusions use certified route-query results. Route checks share the realization ω and scope encounter sets $N_\sigma(\pi, \omega)$.

Rule	Condition and consequence	Evidence
Instruction contract consistency		
Source support	Retained phrases must occur in u. Qualifier and argument phrases must occur within the descriptions they modify. Sequence markers alone do not establish preferences.	Source phrases and their enclosing descriptions
Event references and scopes	Each event reference must resolve unambiguously. Scope references identify boundary events whose selected occurrences must satisfy $j_a < j_b$.	Referenced events and boundary occurrences
Order consistency	Applicable hard order predicates require the corresponding strict inequalities. Self-orders and strict-order cycles among these orders imply $\perp$. A hard NEXT(e_a, e_b) conflicts with a required event forced between its boundaries.	Order edges, cycles, and intervening events
COUNT and visits	An aggregate COUNT phrase alone does not introduce a VIA event. A counted entity becomes a requested visit only when independently specified.	COUNT phrase and visit source phrase
Scene grounding and contract consistency		
Type and lexical matching	Compatible entity and relation types restrict admissible matches. A unique admissible type determines the entity or relation kind. An unambiguous match to the ASG vocabulary determines the corresponding label match.	Entity kinds, relation signatures, and scene vocabulary
Spatial filtering	$\text{IN}(e_i, n_y) \wedge n_x \notin n_y \Rightarrow n_x \notin N_i$. The defined *near*, *nearest*, *farthest*, and portal *between* predicates similarly filter candidates.	Membership in M and geometry Γ
Identity and local binding	Explicit coreference requires $\beta(e_i) = \beta(e_j)$. For a required event, $N_i = \varnothing$ blocks binding. A singleton domain fixes its target, subject to coreference consistency.	Candidate domains and explicit coreferences
COUNT feasibility	For a hard COUNT clause with $\bowtie \in \{=, \geq\}$, $k > \lvert N_C \rvert$ implies $\perp$ under the current grounding.	Requested count and grounded entity identities
Contract preservation	Grounded and executable constraints must reflect the current judgments. When those judgments change, the affected constraints are recomputed.	Current and stored contract, grounding, and executable fields
Global binding determination		
Hard feasibility	Certified $\mathcal{F}_\beta = \varnothing$ excludes β from $\mathcal{B}_H$. Establishing $\mathcal{B}_H = \{\beta\}$ determines the binding.	Replayed feasible routes and infeasibility certificates
Preference selection	Certified minima $p^*(\beta)$ determine $\mathcal{B}_*$. If $\mathcal{B}_* = \{\beta\}$, select that binding. Tied minima retain alternatives.	Preference costs and optimality evidence
Complete-route verification		
Visits and orders	Replay checks $q_{j_i} \in D(\beta(e_i))$, portal crossings, the terminal visit, and applicable order predicates under one ω. A failed required check rejects the route.	Visit witnesses, crossings, and waypoint indices
Scoped hard constraints	For clauses in H, require $N_\sigma(\pi, \omega) \cap N_A = \varnothing$ and $\lvert N_\sigma(\pi, \omega) \cap N_C \rvert \bowtie k$. A violation rejects the route.	Scope boundaries and encountered entity identities
Soft constraints	Each violated clause in P incurs its cost once. Their sum gives $p(\pi, \omega; P)$, including the cost of any unrealized preferred visit.	Clause satisfaction and violation costs

Evidence from connected derivations. Let $\mathcal{T}$ contain the current rule applications, with $\mathrm{prem}(d)$ and $\mathrm{out}(d)$ denoting the premise set and conclusion of application d. For implicated judgments S, define their directly connected derivations as

$$\mathcal{E}(S) = \{d \in \mathcal{T} : S \cap (\mathrm{prem}(d) \cup \{\mathrm{out}(d)\}) \neq \varnothing\}. \tag{11}$$

For the conflict above, $S = \{K_1, K_2\}$. Either judgment may occur as a premise or conclusion in either rule form of Equation 5. Incoming derivations expose the support for a judgment; outgoing derivations expose its consequences and further conflicts. Following these links supplies additional upstream support and downstream dependencies when needed. A predicted premise with no incoming derivation retains its instruction phrase or observation as its source.

Repair evidence combines these rule relations with source phrases, relevant scene facts, the failed check, and admissible alternatives. A count diagnostic links the requested number to the grounded identities and the filters that produced that set. A route diagnostic links an offending encounter to its active scope and clause. The VLM receives implicated semantic fields, current interpretations and grounding, diagnostics, and legal label alternatives.

Determined and predicted repairs. For a demonstrated blocking conflict, the engine removes the judgments selected for revision, Δ^-, and their dependent conclusions. Rules derive replacements when the remaining judgments uniquely determine them. Otherwise, the VLM proposes revisions to the implicated semantic clauses or label correspondences using the original instruction and repair evidence. Both kinds of replacements form Δ^+ in Equation 8 and are checked after the update. Backend failures, inconclusive queries, warnings, and preference violations alone do not trigger semantic repair.

Invalidation and replay. Changing a semantic match can change a candidate domain, invalidate binding evidence, and alter the resulting route. The engine retains unaffected judgments and recomputes affected grounding, binding queries, and route checks. Maintaining supports and revising dependent beliefs follows the truth-maintenance principle (Doyle, 1979). Scene and route queries supply task-specific evidence.

B.3 Global Route Queries

Route queries supply evidence for target determination, verification of adopted routes, and diagnosis of conflicts under the current contract. For a fixed binding, the solver augments G_{nav} with the start and target positions. Within regions, collision-free visibility edges connect navigable positions; portal and stair transitions connect regions. The search domain is finite, and physical edges are evaluated together with task progress.

State and transitions. A conceptual search state is

$$z = \big(x, \eta, \{C_\sigma\}_{\sigma \text{ active}}, \rho\big), \tag{12}$$

where x is the current navigation vertex, η records requested-event and order/scope progress, C_σ stores the encountered identities needed by each scope's count clauses, and ρ tracks preference satisfaction and incurred penalties. Selected event occurrences are recovered with the route.

Traversing an edge produces an ordered trace of entity entries and portal crossings over the whole segment. The monitor updates events, scopes, identity sets, and preferences in trace order using Appendix A. Requested occurrences must meet their visit and ordering conditions; incidental encounters still affect scoped clauses. Hard avoidance violations and exceeded count upper bounds reject transitions immediately. Lower bounds and count equality are checked when the scope closes. A terminal state must realize the required events, close their scopes, and end at the bound goal. Unrealized preferred visits incur their declared costs.

Route monitoring. The task state summarizes the physical trace and selected event occurrences from START through the current vertex. Each edge update preserves this summary under the same visit, scope, and count semantics, so an accepting state satisfies the grounded hard requirements. Identity sets prevent revisits from increasing distinct-entity counts.

Temporal monitors implement finite-trace visit and order patterns (De Giacomo & Vardi, 2013), supplemented with identity-set memory for counts. Z3 (de Moura & Bjørner, 2008) separately checks order and scope consistency using integer event positions. These symbolic checks can reject inconsistent contracts before physical route search.

Feasibility and optimality evidence. A replay-checked witness establishes feasibility. Infeasibility requires exhausting the relevant finite search or a valid certificate from a relaxation. Connectivity and region-topology relaxations can exclude an assignment when even the relaxed problem has no route; a route in a relaxation alone does not establish physical feasibility.

Each preference contributes a nonnegative cost at most once. A zero-cost witness therefore certifies the minimum. A positive minimum requires completed optimization or a matching lower bound excluding smaller costs. Interrupted search without such evidence returns an unknown result for the unresolved claim. Predecessor links reconstruct waypoint segments and event occurrences; replay evaluates the complete route against the contract. These certificates concern the represented search domain.

B.4 Worked Determination Example

Figures 2 and 3 use one recorded run with the instruction:

> Go directly to the bedroom, avoiding the hallway on the upper floor before reaching the bedroom, then go directly to the chair with a nightstand nearby.

Verification and semantic repair. The initial contract contains the correct avoidance scope $\sigma = (\mathrm{START}, v)$ and an incorrect self-order $\mathrm{BEFORE}(v, v)$. The strict-order rule gives $j_v < j_v$, so verification blocks before dependent route queries. Using the source phrase, VLM repair removes this order while preserving the visits, avoidance clause, and preferences. The remaining order list is empty; the required visit still precedes the terminal goal through their event roles. The failed rule localizes the inconsistency, while the source phrase guides its semantic correction.

Replayed target determination. Replayed grounding yields bedroom regions R4/R9/R10, avoided hallway R7, and goal chair 50 in R10. Let β_i bind v to Ri with the chair fixed. The feasible prefix for β_4 is R5–R0–R1–R4. R7 separates the start from R9 and R10 in the region topology, so reaching either candidate before v contradicts the avoidance requirement. Hence

$$\mathcal{B}_{\mathrm{sat}} = \{\beta_4\}, \qquad \mathcal{B}_{\mathrm{unsat}} = \{\beta_9, \beta_{10}\}, \qquad \mathcal{B}_{?} = \varnothing.$$

Equation 9 therefore determines R4. The rejected rows in Figure 3(b) show necessary topological passages, while the R4 row shows a feasible route prefix.

Complete-route verification. The full region sequence is R5–R0–R1–R4–R1–R0–R7–R10. Replay checks that R7 is encountered only after v, when its avoidance scope has ended. This run uses a ground-truth ASG. The RGB-D views illustrate the corresponding scene.

C ASG Construction

Our ASG instantiation uses a registered pre-exploration RGB-D scan. MaskClustering (Yan et al., 2024) supplies 3D object instances. VLM recognition uses mask-conditioned RGB views following OpenVoxel (Huang et al., 2026), retaining surrounding context while highlighting the intended instance. Floor and wall geometry partition regions, which VLMs label using views and contained objects.

Measured wall openings and SAM 2 masks (Ravi et al., 2025) propose portals; identified stairs propose cross-level transitions. Geometric checks establish passage interfaces and navigable anchors. Containment supplies membership, and portal incidence supplies region connectivity. Entity encounter geometry is retained separately from collision geometry, allowing an object neighborhood to affect a task constraint without becoming a physical obstacle.

D Evaluation Details

D.1 Benchmark Construction

Table 1 compares benchmarks and gives two independent partitions by task family and how the target is determined. Explicit preferences occur in 28.13% of tasks. Other overlapping properties appear in Section 4.1.

Tasks are constructed from scene entities, candidate descriptions, and route constraints. Symbolic validation retains a feasible route witness and checks how local qualifiers, hard constraints, or preferences resolve the target. Candidate diagnostics concern the alternatives remaining after semantic and spatial matching. Constraint annotations and these diagnostics are available only to the evaluator.

The nine HM3DSem scenes contribute 26, 33, 18, 30, 9, 10, 27, 27, and 12 tasks, respectively, for scene IDs 00824, 00829, 00843, 00848, 00849, 00853, 00861, 00873, and 00891. Natural-language instructions are generated from the symbolic tasks using Gemini-3.7-Flash (109) (Google DeepMind, 2026b) and GPT-5.6-Sol (83) (OpenAI, 2026). The 89 tasks whose targets are determined by constraints or preferences contain 42 preference tasks. The 103 control tasks contain 12.

Task settings. The benchmark restricts GOAL to objects or regions and uses a $1\,\mathrm{m}$ threshold for object approach and encounter. Each violated preference contributes one unit of cost, at most once. For a fixed binding, equal-penalty routes are ranked by length. Length does not resolve ties between target bindings. Appendix A.1 defines the spatial qualifiers used in target descriptions.

D.2 Scoring and Budgets

Evaluation checks the physical waypoint trace, including intermediate encounters, event order, scoped avoidance, and distinct-entity counts. Full IS and Hard NS are task success rates. Prefer first computes the satisfied fraction of explicit preference clauses for each applicable task, then averages across those tasks. Preference satisfaction is scored independently of hard-constraint success. Preference-free tasks contribute to Full IS / Hard NS but not Prefer.

Main comparison. Main-table quality uses fixed denominators of 178 supported native-interface tasks and 192 ASG-based tasks (including Direct and Direct-Feedback), with 51 and 54 preference tasks. Compute-budget failures and runtime exceptions contribute zero. Run completion rates divide by all 192 issued tasks. Unsupported starts, budget failures, and runtime exceptions are not counted as completed runs. Completed runs include incorrect decisions and native step-limit terminations. Table 7 gives the outcome counts.

Table 7: **Run outcomes.** Each row sums to 192. Resource denotes token/solver-budget exhaustion. Error denotes a runtime exception.

Method	Completed	Resource	Error	Unsupported
NavGPT	177	1	0	14
MapGPT	176	2	0	14
CA-Nav	178	0	0	14
GC-VLN	178	0	0	14
SNav	178	0	0	14
DualVLN	178	0	0	14
NavGPT+ASG	191	1	0	0
GC-VLN+ASG	192	0	0	0
Lang2LTL-2+ASG	192	0	0	0
LIMP+ASG	190	2	0	0
Direct	191	1	0	0
Direct-Feedback	190	2	0	0
ReCoNav	191	1	0	0

Hard-constraint-driven target resolution. The Hard44 analysis covers the 44 tasks in which hard constraints determine the target and whose starts are supported by all methods: 14 avoidance,

22 counting, and eight composite tasks. Full IS, Hard NS and run completion divide by 44. Prefer divides by the four preference-containing tasks. Final budget failures and exceptions score zero. All methods are evaluated on the same 44 task IDs.

Ablation accounting. Component ablations use all 192 tasks for Full IS and Hard NS, and all 54 preference tasks for Prefer. Budget failures and exceptions contribute zero. The Forced-Early subgroup analysis uses all 89 resolving tasks and all 103 control tasks.

Model usage and time. Token and time means use completed runs. Token counts cover instrumented generative calls, counting reasoning tokens once. They exclude non-generative perception and policy computation. Uninstrumented usage is marked n/a. Shared upstream calls are attributed to each ablation that requires them.

D.3 Baselines and Runtime

NavGPT reasons over textual observations and navigation history, while MapGPT incorporates an online topological map into its prompts (Zhou et al., 2024; Chen et al., 2024). CA-Nav uses object, location, and direction constraints as completion criteria for switching between subinstructions. GC-VLN solves geometric constraints among waypoints and objects to determine waypoint positions (Chen et al., 2025; Yin et al., 2025). SNav is fine-tuned on navigation data requiring spatial perception and reasoning. DualVLN combines a VLM for waypoint prediction with a diffusion policy for local execution (Yang et al., 2026; Wei et al., 2026).

Original-interface baselines use their released decision procedures with simulator and scoring adapters. The +ASG variants additionally receive scene entities, relations, and the navigation graph: NavGPT uses scene context and graph connectivity; GC-VLN uses ASG entities and geometry in its spatial constraint graph; Lang2LTL-2 and LIMP retain their specification and planning procedures with the graph as an execution backend.

Simulated agents have radius 0.25 m and height 1.50 m. Native interfaces use a size-matched Habitat navmesh. ASG-based methods use G_{nav}. The backends differ in discretization and support 178 and 192 exact starts, respectively. SNav and DualVLN retain their trained checkpoints. Lang2LTL-2 retains its T5 translator and embedding model.

External-method generative calls and ReCoNav's Qwen variants use Qwen3.8-27B-FP8 with thinking disabled. ReCoNav and its ablations allow two feedback/repair rounds, 90 s solver calls, and a 360 s binding-resolution budget. External-method services run on RTX 4090 GPUs. Some methods share an inference endpoint. External timing includes task initialization. ReCoNav timing excludes service startup and evaluation.

Unresolved bindings. After the semantic repair budget, a remaining ambiguity uses a seeded choice among assignments not excluded by the available evidence. When preference minima are completely certified, this choice is restricted to tied minimizers. Forced-Early and ReCoNav share this execution fallback, which is recorded separately from rule-determined binding. The chosen assignment must still yield a feasible route.

D.4 Additional Ablation Results

All variants share the task population and GT-ASG within each backbone. Direct and Direct-Feedback use the same contract solver as ReCoNav. Direct-ID-Sequence predicts target entity IDs in visit order and asks the solver to connect these targets without enforcing route clauses. The comparison with ReCoNav changes both the task representation and the reasoning procedure. Forced-Early selects targets after semantic and spatial matching, before the solver checks route constraints. It omits global feasibility and preference queries for target determination while retaining repair, final route checks, and the shared unresolved-binding fallback. No Verify/Repair removes the additional verification and evidence-guided repair stage, including the resulting revision and recomputation, while retaining parser validity and route-feasibility checks.

Table 8 compares Forced-Early and ReCoNav on 89 tasks whose targets are determined by hard constraints or preferences and 103 control tasks whose targets are identified by local qualifiers or

direct references. Full IS gains concentrate on the resolving tasks. Tables 9 and 10 report additional quality and latency metrics.

Table 8: **Results by how the target is determined.** Entries are Full IS successes / tasks. Both variants use all 89 resolving tasks and 103 control tasks. Failures score zero.

	Resolving tasks		Control tasks	
Backbone	Forced-Early	ReCoNav	Forced-Early	ReCoNav
GPT	28/89	**89/89**	103/103	103/103
Gemini	28/89	**88/89**	101/103	101/103
Qwen	22/89	**71/89**	86/103	85/103
Gemma	25/89	**87/89**	98/103	97/103

Table 9: **Additional ablation quality.** Hard NS and Prefer (%) use all 192 tasks and all 54 preference tasks, respectively.

	GPT		Gemini		Qwen		Gemma	
Variant	Hard NS	Prefer (n)	Hard NS	Prefer (n)	Hard NS	Prefer (n)	Hard NS	Prefer (n)
Direct	92.71	79.63 (54)	27.60	18.52 (54)	28.13	22.22 (54)	29.17	24.07 (54)
Direct-Feedback	97.92	90.74 (54)	60.94	47.22 (54)	48.44	38.89 (54)	52.08	44.44 (54)
Direct-ID-Sequence	90.63	80.56 (54)	86.98	77.78 (54)	50.52	59.26 (54)	58.33	65.74 (54)
Forced-Early	81.25	48.15 (54)	79.17	48.15 (54)	67.71	44.44 (54)	75.52	46.30 (54)
No Verify/Repair	98.96	98.15 (54)	96.35	98.15 (54)	77.60	74.07 (54)	93.23	90.74 (54)
ReCoNav	100.00	100.00 (54)	98.44	100.00 (54)	85.42	79.63 (54)	95.83	94.44 (54)

Table 10: **Ablation completion and latency.** Run completion counts use 192 issued tasks per variant. Time means use completed runs.

	GPT		Gemini		Qwen		Gemma	
Variant	Completed	Time (s)	Completed	Time (s)	Completed	Time (s)	Completed	Time (s)
Direct	191	52.5	191	87.6	191	118.5	192	82.3
Direct-Feedback	192	57.4	191	208.5	190	282.4	191	191.8
Direct-ID-Sequence	191	12.8	190	25.1	189	24.5	189	19.2
Forced-Early	192	56.9	191	158.8	191	152.0	191	83.0
No Verify/Repair	192	31.9	192	48.3	191	65.4	191	40.6
ReCoNav	192	32.1	192	52.4	191	100.1	190	44.4

D.5 Real-Scene Visualization Setup

The real-scene visualization uses calibrated RGB images, mesh-derived depth, and camera poses. Regions and portals are manually annotated; object labels are predicted, with the illustrated objects checked against their RGB views. The instruction contract yields a 165-waypoint, 14.53 m planned route that avoids the office and visits the selected bin before the washing machine. Four photographs show the robot at the start, selected bin, laundry doorway, and washing machine.